

MOD-04 Web Dashboard & Speech-to-Text Technical Module Documentation

Module Task Distribution

Name	Student ID	Role	Responsibilities
Ömer Nacar	210104004814	Primary	Communication bridge design, telemetry JSON structure, WebSocket server architecture
Tuana Melisa Aksoy	230104004903	Primary	Offline STT integration using Vosk / Whisper.cpp, speech command parsing
Uğur Anıl Güney	210104004011	Secondary	Video streaming pipeline and dashboard communication support
Fatma Öztürk	230104004152	Secondary	Operator command routing, system integration with other modules

Module Responsibilities Summary

The MOD-04 module is responsible for providing the **communication layer between the robot and the operator interface**.

Its core tasks include:

- Running the **Flask / WebSocket dashboard backend**
- Broadcasting **telemetry data** to the Unity interface
- Streaming **live video frames** from the vision module
- Processing **manual override commands** from the operator
- Running the **offline speech-to-text pipeline**
- Translating voice commands into structured control intents

This module acts as the **central communication hub** between the robot autonomy stack and the human operator interface.

Table of Contents

1. Module Overview
 2. Purpose of the Module
 3. Internal Responsibilities of MOD-04
 - 3.1 Web Dashboard Communication Subsystem
 - 3.2 Offline Speech-to-Text Subsystem
 4. Data Structures Used in MOD-04
 - 4.1 AugmentedStatusReport
 - 4.2 VoiceCommandData
 5. Connections Between MOD-04 and Other Modules
 - 5.1 Connection with MOD-02 (AI & Vision Pipeline)
 - 5.2 Connection with MOD-05 (Unity Digital Twin / Operator Dashboard)
 - 5.3 Connection with MOD-03 (Audio & Navigation)
 - 5.4 Connection with MOD-01 (Embedded & Hardware)
 6. Functional Workflow of MOD-04
 7. Importance of MOD-04 in the System
 8. Risks and Design Considerations
 9. Conclusion
-

Web Dashboard & Speech-to-Text Module

1. Module Overview

MOD-04 (Web Dashboard & STT) is the communication and operator-interaction layer of the system. Its main responsibility is to connect the robot's internal software stack running on the Raspberry Pi 5 with the external operator interface. This module provides a Flask/WebSocket-based backend for telemetry and video streaming, routes operator-issued manual control commands into the robot control flow, and performs offline speech-to-text processing for voice-based control. The module therefore acts as a bridge between autonomous robot functions and human supervision.

In the project architecture, MOD-04 is positioned between the Unity operator dashboard and the internal robot-side modules. It exchanges telemetry, video, and audio with the dashboard, communicates with the vision pipeline to coordinate RAM-heavy execution phases, and forwards operator intervention signals toward the autonomous control logic. This makes MOD-04 a central integration module rather than an isolated processing block.

2. Purpose of the Module

The purpose of MOD-04 is to provide a reliable operator control channel and monitoring interface. While lower-level modules perform hardware control, perception, and navigation, MOD-04 is responsible for making their outputs observable and controllable from outside the robot. In practice, this means:

- sending live robot telemetry to the dashboard,
- sending live video frames to the operator interface,
- accepting manual override commands from the operator,
- receiving voice audio data and converting it into structured robot commands.

This module is especially important because search-and-rescue systems require both autonomy and operator intervention. MOD-04 enables that hybrid behavior by serving as the communication bridge between machine-side autonomy and user-side supervision.

3. Internal Responsibilities of MOD-04

MOD-04 has two main internal subsystems:

3.1 Web Dashboard Communication Subsystem

This subsystem is defined through the `IWebDashboard` interface. It is responsible for starting the backend server, broadcasting telemetry, streaming video, and handling commands sent by the operator. The interface exposes four main methods:

- `start_server(...)` initializes the Flask and SocketIO server.
- `broadcast_telemetry(...)` sends the current robot status to all connected clients.
- `stream_video_frame(...)` transmits compressed JPEG image frames.
- `on_operator_command_received(...)` processes commands coming from the operator and triggers interruption of the autonomous flow if necessary.

This part of the module is the runtime communication gateway of the robot.

3.2 Offline Speech-to-Text Subsystem

This subsystem is defined through the `ISTTEngine` interface. It loads an offline model and converts incoming `.wav` byte streams into structured command data. The interface exposes two methods:

- `load_offline_model(...)` loads the speech recognition model into Raspberry Pi memory.
- `process_audio_blob(...)` converts raw audio bytes into a `VoiceCommandData` result containing transcribed text, detected intent, and confidence score.

This subsystem allows the system to support voice commands without depending on cloud infrastructure, which is important for offline or emergency-field operation.

4. Data Structures Used in MOD-04

Two main data contracts are defined in this module.

4.1 `AugmentedStatusReport`

`AugmentedStatusReport` is the telemetry structure transmitted from MOD-04 to the dashboard. It contains both environmental and mission-related robot status values, including robot position, temperature, smoke presence, victim status, stuck state, rescue priority, and acoustic localization output. These fields allow the dashboard to display not only raw telemetry, but also semantically enriched system status for the operator.

Its fields are:

- `pos_x, pos_y`: robot position
- `temperature`: ambient temperature
- `smoke_detected`: smoke sensor status
- `victim_status`: detected victim condition
- `is_stuck`: whether the robot is stuck
- `priority_level`: rescue urgency
- `acoustic_hit`: whether a sound source was detected
- `acoustic_angle`: estimated direction of the sound source

4.2 `VoiceCommandData`

`VoiceCommandData` is the output of the STT pipeline. It stores the recognized raw text, the interpreted command intent, and the confidence score. This transforms unstructured audio input into machine-usable control information. Example intents include "STOP", "RETURN_HOME", and "ACTIVATE_BEACON".

5. Connections Between MOD-04 and Other Modules

MOD-04 is strongly connected to multiple modules in the system. Its technical role can only be understood clearly by describing these interfaces.

5.1 Connection with MOD-02 (AI & Vision Pipeline)

MOD-04 communicates bidirectionally with MOD-02. The project report defines three important Python calls in this interaction:

- `get_latest_target()`
- `pause_vision_pipeline()`
- `resume_vision_pipeline()`

This means MOD-04 can request the latest vision-based target information from MOD-02 and can also temporarily suspend and restart the vision pipeline. The reason for this coordination is resource management: offline STT is a heavy operation on the Raspberry Pi 5, and the documentation explicitly states that the vision pipeline should be paused before speech processing in order to reduce memory pressure and avoid out-of-memory conditions.

So, the connection between MOD-02 and MOD-04 is not only functional but also performance-driven. MOD-04 depends on MOD-02 for perception data, while MOD-02 may be temporarily controlled by MOD-04 during STT execution.

5.2 Connection with MOD-05 (Unity Digital Twin / Operator Dashboard)

MOD-04 has the most direct external connection with MOD-05. According to the system documentation, telemetry and video are sent from MOD-04 to MOD-05, while operator commands and audio blobs are sent from MOD-05 to MOD-04. Specifically:

- `broadcast_telemetry()` sends telemetry to Unity,
- `stream_video_frame()` sends compressed JPEG frames,
- operator commands are received as JSON payloads,
- `process_audio_blob()` handles incoming audio bytes from Unity.

This makes MOD-04 the backend communication layer for the operator dashboard. MOD-05 is the front-end visualization and control environment, and MOD-04 is the server-side component that powers it. Without MOD-04, MOD-05 would not receive robot status or send control signals back into the system.

5.3 Connection with MOD-03 (Audio & Navigation)

There is no direct API contract shown between MOD-04 and MOD-03 in the interface files, but the project architecture shows that acoustic localization output becomes part of the high-level system state and is eventually included in the telemetry visible through MOD-04. In particular, `acoustic_hit` and `acoustic_angle` are included in the `AugmentedStatusReport`, which indicates that MOD-04 distributes navigation-related acoustic results to the operator dashboard.

Therefore, MOD-04 serves as a dissemination layer for some outputs originating from MOD-03, even if the coupling is indirect.

5.4 Connection with MOD-01 (Embedded & Hardware)

MOD-04 does not directly define a low-level embedded protocol with MOD-01 in its own Python interfaces, but it ultimately depends on telemetry values that originate from the embedded layer, such as temperature, smoke detection, and stuck information. These hardware-

level values are first produced by MOD-01 and then propagated upward through the system until MOD-04 packages them into the dashboard telemetry structure.

Thus, MOD-04 is not a producer of raw sensor data, but it is the module responsible for presenting that data to the operator in an accessible and actionable form.

6. Functional Workflow of MOD-04

A typical runtime flow involving MOD-04 can be summarized as follows:

1. The Flask/WebSocket server is started on the Raspberry Pi.
2. Robot-side data coming from perception, navigation, and embedded layers is assembled into an `AugmentedStatusReport`.
3. This report is broadcast to Unity through WebSockets.
4. At the same time, compressed video frames are streamed to the dashboard.
5. If the operator sends a manual override command, MOD-04 receives it and routes it into the control system.
6. If the operator sends push-to-talk audio, MOD-04 invokes the STT engine.
7. Before STT processing, the vision pipeline may be paused.
8. Audio is converted into `VoiceCommandData`.
9. After speech processing finishes, the vision pipeline is resumed.

This sequence shows that MOD-04 is an event-driven integration module coordinating real-time data exchange and operator interaction.

7. Importance of MOD-04 in the System

MOD-04 is critical because it makes the rest of the robot system observable and controllable. Modules such as embedded control, vision, and acoustic navigation can function autonomously, but MOD-04 provides the human-facing bridge required for supervision in emergency scenarios. It also improves operational flexibility by allowing both graphical dashboard control and voice command control. The module therefore supports transparency, intervention, and situational awareness, which are all essential in a rescue robotics context.

8. Risks and Design Considerations

The main technical concern associated with MOD-04 is memory and performance pressure on the Raspberry Pi 5. The documentation states that running vision inference and STT simultaneously may cause severe RAM stress, which is why `pause_vision_pipeline()` and

`resume_vision_pipeline()` exist. This means the module design already anticipates resource contention and includes a coordination strategy to mitigate it.

Another important consideration is that MOD-04 relies on WebSocket event naming and operator command payload definitions that must remain consistent with the Unity side. Any mismatch between backend and frontend contracts would affect communication reliability. The README also notes that some WebSocket constants are still marked as TODO, which means this interface requires careful final integration testing.

9. Conclusion

In conclusion, MOD-04 is the communication and interaction hub of the project. It connects the robot's internal intelligence and sensing pipeline with the operator-facing Unity dashboard. Its dashboard subsystem streams telemetry and video, its command subsystem accepts manual interventions, and its STT subsystem converts voice into actionable robot commands. Through its connections with MOD-02 and MOD-05, and its indirect dependency on MOD-01 and MOD-03 outputs, MOD-04 plays a central role in integration across the entire system. For this reason, it should be documented not only as a standalone module, but also as the module that enables coordination between autonomy and operator control.